Bachelor Thesis

April 10, 2026

Attribute–Embedding Correlation

A distribution divergence approach on Filtered Vector Search

Matteo Iulian Adam

of University of Zurich, Switzerland (23-744-519)

supervised by

Prof. Dr. Yannis Chronis, ETH
Prof. Dr. Dan Olteanu, UZH
Dr. Mageirakos Vasilis, ETH

University of Zurich[UZH]

Bachelor Thesis

Attribute–Embedding Correlation

A distribution divergence approach on Filtered Vector Search

Matteo Iulian Adam

University of Zurich[UZH]

Bachelor Thesis

Author: Matteo Iulian Adam, matteoiulian.adam@uzh.ch

URL: https://github.com/johndonneUZH/thesis

Project period: 2026-01-15 - 2026-07-01

Systems Group, Institute for Computing Platforms
Department of Computer Science (D-INFK), ETH Zurich

Acknowledgements

Abstract

Contents

1 Introduction **1**
1.1 Motivation and Problem Setting . . . 1
1.2 Research Gap . . . 1
1.3 Research Objectives & Questions . . . 1
1.4 Thesis Structure . . . 2

2 Background and related work **3**
2.1 Vector Search . . . 3
2.1.1 Vector representation . . . 3
2.1.2 Nearest Neighbor Search . . . 3
2.1.3 Approximate Nearest Neighbor Search . . . 3
2.1.4 Metrics for ANNS Evaluation . . . 4
2.2 Filtered Vector Search . . . 5
2.2.1 Selectivity . . . 5
2.2.2 Correlation . . . 6
2.3 Execution Strategies . . . 7
2.3.1 Pre-filtering . . . 7
2.3.2 Post-filtering . . . 7
2.3.3 Inline Filtering . . . 7

3 Methodology **9**
3.1 Data Model . . . 9
3.2 Query Model . . . 9
3.3 Evaluation Setup . . . 9
3.4 Objective . . . 9

4 Dataset Construction **11**
4.1 Design Goals . . . 11
4.2 Correlated Attribute Generation . . . 11
4.2.1 Clustering . . . 11
4.2.2 Cluster Anchors . . . 11
4.2.3 Gaussian Weighting . . . 12
4.2.4 Soft Assignment . . . 13
4.2.5 Radial Basis Function Interpretation . . . 13
4.2.6 Continuous Attribute Construction . . . 14
4.3 Uncorrelated Attribute Generation . . . 14
4.4 Query Range Generation . . . 14
4.4.1 Positive (correlated) . . . 14
4.4.2 Negative (anti-correlated) . . . 14
4.4.3 Random . . . 14
4.5 Selectivity Control . . . 14
4.6 Dataset Validation . . . 14
4.6.1 Neighbor consistency . . . 15
4.6.2 Cluster distributions . . . 15

5 Correlation Metric **17**
5.1 Local vs Global Attribute Distributions . . . 17
5.2 Cluster-based Approximation . . . 17
5.3 Jensen–Shannon Divergence . . . 17
5.4 Lift (range alignment) . . . 17
5.5 Combined Score & Interpretation . . . 17

6 System & Execution Strategies **19**
6.1 Pre-filtering . . . 19
6.2 Post-filtering . . . 19
6.3 Inline Filtering . . . 19
6.4 Implementation Details . . . 19

7 Experimental Setup **21**
7.1 Offline computed statistics . . . 21
7.2 Evaluation Metrics . . . 21
7.3 Ground Truth Computation . . . 21

8 Results & Discussion **23**

Bibliography **25**

A First Appendix **27**

B Second Appendix **29**

1. Introduction

1.1 Motivation and Problem Setting Consider a user searching for a shirt similar to a high-end item, but with a price below a given threshold (e.g., under 30). This query combines two components: similarity, to find visually or stylistically related items, and filtering, to enforce a constraint on structured attributes such as price.

This type of query is representative of modern applications, where vector search is combined with structured predicates. In such settings, the goal is not only to retrieve similar items, but to do so within a restricted subset of the data.

Processing these queries efficiently is challenging. While vector search techniques exploit the geometric structure of the embedding space, filtering restricts the search to a subset of the data. This restriction not only reduces the number of candidate points, but can also change their distribution in the embedding space.

A key factor is how the filtered vectors are positioned relative to the query. In some cases, the filter selects vectors that are concentrated near the query, making them easy to retrieve. In other cases, they are located farther away, requiring the search to explore less relevant regions of the space.

This difference reflects how structured attributes interact with the embedding space. When attributes and embeddings are aligned, filtering preserves the local neighborhood of the query. When they are not aligned, filtering disrupts this structure and makes the search more difficult.

Capturing this effect is essential for efficient query processing, as it directly influences how different execution strategies behave and how much work is required to retrieve relevant results.

1.2 Research Gap Existing work, however, typically handles this interaction only indirectly and in a system-specific manner. In particular, most evaluations rely on synthetic workloads where attributes are generated independently of the vector data.

Starting from NHQ [1], prior work constructs such workloads by attaching synthetic attributes to vectors, following approaches used in systems such as Milvus [2] and Multiattribute ANNS [3]. In these settings, attribute values are assigned by randomly sampling from predefined categories. While this enables controlled experiments, it ignores any connection between attributes and embeddings. Similarly, Filtered-DiskANN [4] generates labels independently of the vector space, explicitly removing any correlation.

As a result, current approaches do not capture how filters and vectors interact in practice, and this effect remains largely unmodeled.

1.3 Research Objectives & Questions The main objective is to treat this relationship as a property of the data and to define a quantitative measure that captures how filter predicates relate to the vector distribution. This enables a more systematic understanding of how filtering affects nearest neighbor search.

This leads to the following research questions:

- How can the relationship between filter predicates and the vector space be formally defined and measured?

- How does this relationship affect the behavior and performance of hybrid search methods?
- How can this information be used to improve query processing beyond assumptions of independence?

The goal is to develop a general and system-independent framework that can be applied across different index structures and query execution methods, including both exact and approximate search.

1.4 Thesis Structure The remainder of this thesis is organized as follows.

Chapter 2 provides the necessary background and reviews related work. It introduces vector search, approximate nearest neighbor methods, and filtered vector search, including key concepts such as selectivity, correlation, and common execution strategies.

Chapter 3 formalizes the problem setting by defining the data model, query model, and the objective of the study.

Chapter 4 describes the construction of the datasets used in this thesis. In particular, it explains how both correlated and uncorrelated attributes are generated, as well as how query ranges and selectivity are controlled.

Chapter 5 introduces the proposed correlation metric. It presents the comparison between local and global attribute distributions, the use of Jensen–Shannon divergence, and the incorporation of lift to capture query-dependent effects.

Chapter 6 describes the system setup and the execution strategies considered, including pre-filtering, post-filtering, and inline filtering, along with implementation details.

Chapter 7 outlines the experimental setup, including evaluation metrics and the computation of ground truth.

2. Background and related work

2.1 Vector Search

2.1.1 Vector representation The representation of complex items (such as text, images, or user behavior) as high-dimensional vectors has become increasingly important in modern data-intensive applications. These vectors, commonly referred to as embeddings, are typically learnt using machine learning models and capture semantic or structural relationships in the data [5]. As a result, semantically similar items are mapped to nearby points in a continuous vector space, enabling similarity to be measured using geometric distance functions such as cosine similarity or Euclidean distance.

The widespread adoption of this paradigm has led to the emergence of vector databases, which extend traditional database systems by enabling efficient similarity-based retrieval. Unlike conventional databases that operate on structured attributes using exact or range-based predicates, vector databases are designed to efficiently retrieve similar items from large collections of vectors by leveraging specialized index structures (e.g., graph-based or tree-based indexes) and approximate nearest neighbor search algorithms [6].

2.1.2 Nearest Neighbor Search Given a dataset of vectors $\{x_1, \ldots, x_n\} \subset \mathbb{R}^d$, the nearest neighbor search problem consists of identifying, for a query vector $q \in \mathbb{R}^d$, the vectors in the dataset that are most similar to q. This problem is also known as the *post office problem*, where one seeks the closest facility to a given location. In the k-nearest neighbor (kNN) setting, the goal is to retrieve the k most similar vectors.

To formalize this notion of similarity, we use a function $D(\cdot, \cdot)$ that assigns a numerical score to a pair of vectors, reflecting how far apart they are in the underlying space. Common choices include Euclidean distance, cosine similarity, and inner product-based measures, each capturing different notions of similarity and directly influencing retrieval quality.

Given such a function, a natural solution is to compute $D(q, x_i)$ for every vector x_i in the dataset and return the closest ones. This approach guarantees exact results, but requires scanning the entire dataset and performing a computation of cost $\mathcal{O}(d)$ per vector, resulting in a total time complexity of $\mathcal{O}(n \cdot d)$. As both the number of vectors n and the dimensionality d grow, this linear scan becomes computationally expensive and leads to high query latency [7]. Moreover, in high-dimensional spaces, the contrast between nearby and far-away points tends to diminish, making it harder to distinguish meaningful neighbors [8].

These limitations motivate the use of approximate nearest neighbor approaches.

2.1.3 Approximate Nearest Neighbor Search Approximate nearest neighbor (ANN) methods trade exactness for efficiency by exploring only a subset of the dataset while aiming to preserve high recall. Instead of exhaustively comparing a query against all data points, ANN algorithms rely on specialized index structures and heuristics to efficiently identify promising candidates.

A prominent class is based on hashing techniques, such as locality-sensitive hashing (LSH), which project high-dimensional vectors into lower-dimensional hash spaces in a way that preserves similarity [9], [10]. The key idea behind LSH is to use hash functions that increase the probability of collision for similar vectors while reducing it for dissimilar ones. This enables efficient retrieval of candidate neighbors from

hash buckets, often achieving sub-linear query time. The downside is that LSH-based methods may require multiple hash tables and can suffer from reduced accuracy or increased memory usage in high-dimensional settings.

Graph-based indexing methods have demonstrated particularly strong empirical performance. In particular, the Hierarchical Navigable Small World (HNSW) graph arranges data points in a multi-layer structure and uses greedy search to quickly reach nearest neighbors [11]. Due to its strong balance between efficiency and accuracy, HNSW has become widely adopted in modern vector search systems, but it typically requires the entire index to reside in main memory. For larger, disk-resident datasets, approaches such as DiskANN instead rely on a single-layer graph (Vamana) that is better suited for disk-based access, avoiding the multi-layer traversal of HNSW and enabling efficient search at billion scale [12].

Recent work has also explored learning-based approaches, where models are used to guide how the search structure is built or traversed [13], [14]. Instead of relying only on manually designed heuristics, these methods learn from the data how to partition the vector space or which regions are more likely to contain relevant neighbors. This makes the search process more efficient and highlights that ANN performance depends not only on the search algorithm, but also on how the data is organized.

In this thesis, ANN search is implemented using the FAISS library [15]. It provides a unified framework that implements a variety of ANN techniques, including graph-based methods such as HNSW, as well as exact search for ground truth computation.

2.1.4 Metrics for ANNS Evaluation ANN systems are typically evaluated along three main dimensions: retrieval quality, query efficiency, and resource usage, reflecting the trade-offs inherent in approximate search [15], [16].

A primary measure of retrieval quality is *recall@k*, which quantifies how many of the true nearest neighbors are retrieved by the approximate method. Let R_k^* denote the set of the k exact nearest neighbors and R_k the set returned by the ANN method. Then:

$$\text{recall@k} = \frac{|R_k \cap R_k^*|}{|R_k^*|}.$$

More generally, n-recall@k measures how many of the true nearest neighbors are successfully retrieved by the approximate method. Concretely, it is the fraction of the top-n ground truth nearest neighbors that appear among the top-k approximate results, where $n \leq k$ [15]. A common choice is $n = k$, in which case the metric reflects how well the approximate result set matches the exact one.

The definition of recall@k varies across the literature: some works use 1-recall@k, focusing only on the nearest neighbor, while others use k-recall@k, which evaluates the retrieved set as a whole. In this thesis, we use the latter.

Query efficiency is typically measured in terms of latency and throughput (queries per second, QPS). Latency captures the time required to answer a single query, while throughput reflects the number of queries that can be processed per unit time. Together, these metrics determine how well a system scales under different workloads.

Resource usage further influences these metrics, including memory consumption, index construction time, and, in disk-based settings, I/O cost during search [15]. For example, graph-based indexes require additional memory to store connectivity information, while compression techniques reduce memory usage at the cost of lower recall.

2.2 Filtered Vector Search Let (X, d) be a metric space, where $X \subseteq \mathbb{R}^d$ is a finite set of vectors and $d(\cdot, \cdot)$ is a distance function (e.g., Euclidean or cosine distance). Each vector $x \in X$ is associated with structured attributes $a(x) \in \mathcal{A}$, where $\mathcal{A}$ denotes the attribute space.

A *predicate* is a Boolean function

$$p : \mathcal{A} \to \{0, 1\},$$

which defines a subset of vectors

$$X_p := \{x \in X \mid p(a(x)) = 1\}.$$

Given a query vector $q \in \mathbb{R}^d$ and an integer k, the filtered k-nearest neighbor problem is to retrieve the k closest vectors to q among those that satisfy the predicate:

$$R_k(q, p) = \text{top-}k \text{ elements of } X_p \text{ with respect to } d(q, \cdot).$$

The defining characteristic of filtered vector search is that the query is evaluated over the subset X_p, rather than over the full dataset X. That is, the search domain changes from (X, d) to (X_p, d). This restriction does not only reduce the number of candidate points; it changes the distribution and structure of the data on which the search is performed.

Formally, if μ denotes the empirical distribution over X, then restricting to X_p corresponds to conditioning on the predicate:

$$\mu_p(x) = \mathbb{P}\big(x \mid p(a(x)) = 1\big).$$

In general, μ_p can differ significantly from μ. For instance, X_p may be much smaller, unevenly distributed, or composed of points that are far apart in the embedding space.

In the full dataset X, the points closest to q may not satisfy the predicate and be excluded from X_p. Thus, the nearest neighbors of q within X_p can be entirely different from the nearest neighbors in X.

Each predicate p defines therefore a different search problem. The subset X_p can vary in size, distribution, and connectivity, and these variations determine how difficult the search is. As a result, filtered vector search is inherently *query-dependent*: instead of operating on a single fixed dataset, the system must handle many different subsets defined at query time.

This creates a fundamental challenge. Most NN methods are designed under assumptions about the global structure of X. When applied to a subset X_p that does not follow the same structure, these methods may become inefficient or lose accuracy. [1], [17]–[19].

2.2.1 Selectivity Selectivity is a fundamental concept in query processing that quantifies the fraction of a dataset satisfying a predicate. Given a dataset X and a predicate p, the selectivity of p is defined as

$$\text{sel}(p) = \frac{|X_p|}{|X|}.$$

The cardinality of p is given by $|X_p|$.

Cardinality estimation has been extensively studied in the database literature due to its central role in query optimization. Classical approaches often assume attribute independence, approximating the selectivity of a conjunction of predicates as the product of their individual selectivities. While computationally convenient, this assumption can lead to large estimation errors in the presence of correlated attributes [20]. Such estimates, combined with statistics such as histograms or samples, are used by query planners to select execution strategies, and inaccuracies can lead to suboptimal plans [21], [22].

In filtered vector search, selectivity determines the size of the candidate set. For high selectivity, $|X_p|$ remains large and the search space closely resembles the full dataset. For low selectivity, $|X_p|$ is small, reducing the number of candidates but restricting the available neighbors.

However, selectivity alone does not fully characterize query difficulty. Two predicates p_1, p_2 with $\text{sel}(p_1) = \text{sel}(p_2)$ may induce subsets X_{p_1} and X_{p_2} with very different structures in the embedding space, leading to different retrieval behavior. In this thesis, we therefore study queries across varying selectivity levels while explicitly accounting for attribute–embedding correlation.

2.2.2 Correlation In addition to the size of the admissible set X_p, performance also depends on how the predicate-induced subset is arranged in the embedding space relative to the query.

To formalize the interaction between the predicate and the embedding space, define

$$g_k(q, S) = \frac{1}{k} \sum_{x \in \mathrm{NN}_k(q,S)} d(q, x),$$

where $\mathrm{NN}_k(q, S)$ denotes the k nearest neighbors of q in a set S, and d is the distance function used for retrieval. Thus, $g_k(q, X_p)$ is the average distance from q to its k nearest admissible neighbors under predicate p [17].

To determine whether these admissible neighbors are unusually close or unusually far, we compare them to a baseline in which the predicate has no geometric structure in the embedding space. Let

$$R_p \subseteq X$$

be a random subset of size $|X_p|$, sampled uniformly from X. Then R_p represents a hypothetical subset with the same cardinality as X_p, but without any predicate-specific spatial organization.

We define the query-level correlation of (q, p) as

$$C(q, p) = \mathbb{E}[g_k(q, R_p)] - g_k(q, X_p),$$

where the expectation is taken over the random draw of R_p.

This quantity admits a natural interpretation:

$$C(q, p) > 0 \quad \Longrightarrow \quad g_k(q, X_p) < \mathbb{E}[g_k(q, R_p)],$$

meaning that the admissible neighbors in X_p are, on average, closer to the query than would be expected under a random subset of the same size. In this case, the query exhibits *positive correlation*.

Conversely,

$$C(q, p) < 0 \quad \Longrightarrow \quad g_k(q, X_p) > \mathbb{E}[g_k(q, R_p)],$$

meaning that the admissible neighbors are farther from the query than expected under the random baseline. In this case, the query exhibits *negative correlation*.

Finally,

$$C(q, p) \approx 0$$

indicates *no correlation*, i.e., the predicate-induced subset behaves, from the perspective of the query, similarly to a random subset of the same size.

Intuitively, positive correlation means that the predicate selects vectors that are geometrically aligned with the query region: the valid answers tend to lie inside or near the query's local neighborhood. Negative

correlation means the opposite: although vectors satisfying the predicate exist, they are systematically displaced from the query's neighborhood, so retrieving them requires exploration beyond the region that would be most promising under vector similarity alone.

This distinction is fundamental for filtered vector search. Two predicates may have the same selectivity, i.e.,

$$|X_{p_1}| = |X_{p_2}|,$$

while inducing very different values of $C(q, p)$. In that case, the two queries have the same number of admissible vectors but differ substantially in geometric difficulty. Correlation therefore captures an aspect of filtered vector search that selectivity alone cannot express.

In this thesis, we develop a practical method to estimate this quantity efficiently.

2.3 Execution Strategies Execution strategies define how structured filter predicates are combined with nearest neighbor search. The main difference between them is when and how the filter is applied in relation to the search process. This design choice directly affects performance, efficiency, and result quality.

In this thesis, we focus on three main execution strategies.

2.3.1 Pre-filtering Let D be a dataset of vectors and ϕ a predicate. Pre-filtering first evaluates the predicate and constructs the subset

$$D_\phi = \{x \in D \mid \phi(x)\}.$$

Nearest neighbor search is then performed only over D_ϕ.

This guarantees that all returned results satisfy the predicate. However, standard vector indexes are built over the full dataset D and cannot be directly restricted to arbitrary subsets D_ϕ. In practice, this often means scanning all vectors in D_ϕ and computing distances explicitly, with cost proportional to $|D_\phi|$. When selectivity is high ($|D_\phi| \approx |D|$), the cost is close to a full scan of the dataset.

2.3.2 Post-filtering Post-filtering performs nearest neighbor search over D, ignoring the predicate. Let $C_k \subseteq D$ denote the top-k retrieved candidates. The predicate is then applied to C_k, and only elements satisfying ϕ are kept.

This allows the index to operate normally, but it does not guarantee that k valid results are obtained. If only a fraction $\text{sel}(\phi)$ of the dataset satisfies the predicate, the expected number of valid results in C_k is approximately $k \cdot \text{sel}(\phi)$. If this number is less than k, additional candidates must be retrieved, increasing the total number of distance evaluations.

2.3.3 Inline Filtering Inline filtering incorporates the predicate into the search itself, so that it directly constrains which nodes are explored. Instead of constructing D_ϕ in advance (pre-filtering) or discarding invalid results after retrieval (post-filtering), the constraint is enforced during traversal.

In graph-based indexes, this amounts to searching over the subgraph induced by the predicate, i.e., nodes $x \in D$ such that $\phi(x)$ holds. A key challenge is that this subgraph is typically not well connected, which can prevent the search from reaching relevant parts of the space.

ACORN [17] addresses this by maintaining connectivity under filtering. At query time, it begins from the standard HNSW entry point and performs greedy routing until it reaches a node satisfying the predicate. From there, the search proceeds only through nodes that satisfy the predicate, with filtering applied to neighbor expansions at each step.

To support this restricted traversal, ACORN provides two variants. ACORN-γ increases connectivity at index construction time by expanding each node's neighborhood by a factor γ, followed by pruning to control index size. This pruning is *predicate-agnostic*, i.e., based solely on standard distance-based heuristics and independent of any predicate, ensuring that the index remains general-purpose while improving connectivity for arbitrary filters. In contrast, ACORN-1 leaves the index unchanged and performs additional neighbor exploration dynamically at query time.

In this thesis, ACORN serves as the representative inline filtering method.

3. Methodology

This chapter describes the methodology used to evaluate the proposed correlation metric. The goal is to study how the relationship between structured attributes and the embedding space affects query behavior in filtered vector search.

To this end, we define a data model and a query model, and evaluate the proposed metric across different datasets and query configurations. The methodology is designed to isolate the effect of attribute–embedding interaction while remaining independent of a specific index structure.

3.1 Data Model We consider datasets of high-dimensional vectors, where each vector is associated with a structured attribute.

As a proof of concept, we use a handcrafted dataset based on SIFT1M, consisting of one million vectors of dimension 128. Structured attributes are generated and attached to these vectors to control the degree of correlation with the embedding space.

In addition, we use real-world embeddings based on GloVe to evaluate the proposed approach under more realistic data distributions.

3.2 Query Model Queries consist of a vector and a structured predicate. The predicate is defined as a range over the attribute domain.

For evaluation, we use a set of 10,000 query vectors. For each query, we retrieve the top-k nearest neighbors under the given predicate, with $k = 100$.

This setup allows us to analyze how filtering affects retrieval behavior under different query conditions.

3.3 Evaluation Setup We evaluate query performance using recall@k, comparing approximate results to exact nearest neighbors.

To isolate the effect of attribute–embedding interaction, we consider queries with varying degrees of correlation between the predicate and the embedding space. This allows us to study how the proposed metric relates to retrieval difficulty and execution behavior.

3.4 Objective

4. Dataset Construction

4.1 Design Goals Our goal is to construct synthetic structured attributes that are going to be later used in our experiments. We generate two types of attributes: (i) correlated attributes, which are explicitly coupled to the geometry of the vector space, and (ii) uncorrelated attributes, which serve as a random baseline.

All attributes are defined over a bounded discrete domain:

$$\mathcal{A} = \{0, 1, \ldots, 200\}.$$

Uncorrelated Attributes In contrast, uncorrelated attributes are generated independently of the vector space. Each vector is assigned a value uniformly at random from $\mathcal{A}$. This removes any relationship between vector similarity and attribute similarity and serves as a baseline to evaluate the effect of correlation.

4.2 Correlated Attribute Generation We aim to enforce a strong relationship between vector similarity and attribute similarity, i.e., vectors that are close in the embedding space should receive similar attribute values.

To achieve this, we construct attributes that vary smoothly over the vector space. We first group similar vectors into clusters and assign each cluster a scalar value, which is then smoothly interpolated to all vectors.

This results in attribute values that are consistent within local regions and change gradually across the space, without sharp cluster boundaries.

4.2.1 Clustering Let $\{x_i\}_{i=1}^{N} \subset \mathbb{R}^d$ denote the set of base vectors.

To capture the geometric structure of the vector space, we first partition the dataset using clustering. Specifically, we apply MiniBatch k-means to obtain k cluster centroids:

$$\{c_j\}_{j=1}^{k}.$$

These centroids serve as representative points that summarize the global distribution of the data.

4.2.2 Cluster Anchors To induce a structured signal over the clusters, we assign each centroid a scalar anchor value. Rather than relying on the arbitrary ordering induced by k-means, we first order the centroids according to their projection onto the first principal component (PC1).

Let π denote the permutation that sorts centroids along PC1. We then define:

$$v_j = \frac{j-1}{k-1}, \quad j \in \{1, \ldots, k\},$$

where c_j follows this ordering.

This ensures that nearby centroids in the embedding space receive similar anchor values, preserving global geometric consistency.

4.2.3 Gaussian Weighting For each vector x_i, we compute its squared Euclidean distance to all centroids:

$$d_{ij} = \|x_i - c_j\|^2.$$

We then define soft assignment weights using a Gaussian kernel:

$$w_{ij} = \exp\left(-\frac{d_{ij}}{2\sigma^2}\right).$$

Bandwidth Parameter The parameter $\sigma > 0$ is referred to as the *bandwidth* of the kernel. It controls how quickly the influence of a centroid decays as a function of distance.

- If σ is small, only very close centroids receive significant weight, resulting in highly localized assignments.
- If σ is large, many centroids contribute similarly, leading to smoother but less discriminative assignments.

Thus, σ determines the scale at which the geometry of the space is observed and directly influences the strength of the induced correlation between vectors and attributes.

Bandwidth Estimation A naive computation of σ based on all pairwise distances $\|x_i - c_j\|$ would require $\mathcal{O}(|S|k)$ operations, which is computationally expensive for large-scale datasets.

To address this, we estimate σ using a random subsample. Let $S \subset \{1, \ldots, N\}$ be a uniformly sampled subset with $|S| = \min(10^4, N)$. We define:

$$\sigma = \text{median}\left(\{\|x_i - c_j\| : i \in S,\ j \in \{1, \ldots, k\}\}\right).$$

Why the Median? We use the median of the sampled distances to estimate σ instead of the mean in order to obtain a robust estimate of the characteristic distance scale.

In high-dimensional spaces, distance distributions often exhibit heavy tails and may contain outliers (e.g., points far from any centroid). The mean is sensitive to such extreme values and can be significantly skewed, leading to an overestimation of σ. This, in turn, would cause the Gaussian kernel to become overly flat, reducing locality and weakening the correlation between geometry and the generated attribute.

In contrast, the median is a robust statistic that is insensitive to outliers and better reflects the typical (central) distance between vectors and centroids. As a result, it yields a more stable and representative bandwidth, ensuring that the kernel maintains an appropriate balance between locality and smoothness.

Monte Carlo Interpretation This estimation can be viewed as a Monte Carlo approximation of the global distance distribution between data points and centroids.

Specifically, instead of computing statistics over the full set of Nk distances, we approximate the median distance by sampling a subset of points uniformly at random. Under this sampling scheme, the empirical distribution of distances over S converges to the true distribution as $|S|$ increases. Therefore, the estimated σ provides a consistent approximation of the characteristic distance scale of the dataset, while significantly reducing computational cost.

4.2.4 Soft Assignment The weights w_{ij} induce a *soft assignment* of each vector to all clusters simultaneously.

In contrast to hard clustering, where each vector is assigned to a single cluster:

$$x_i \mapsto \arg\min_j \|x_i - c_j\|,$$

soft assignment produces a distribution over clusters:

$$x_i \mapsto (w_{i1}, w_{i2}, \ldots, w_{ik}).$$

Each weight w_{ij} reflects the degree to which centroid c_j contributes to the representation of x_i. Nearby centroids receive higher weights, while distant centroids contribute negligibly.

This has two key advantages:

- It avoids discontinuities introduced by hard assignments, where small changes in x_i may abruptly change cluster membership.
- It allows multiple nearby clusters to jointly influence the attribute value, leading to smoother and more robust metadata.

4.2.5 Radial Basis Function Interpretation The Gaussian kernel used in the weighting scheme is a specific instance of a *radial basis function* (RBF). In general, an RBF is a function of the form:

$$\phi(x, c) = g(\|x - c\|),$$

i.e., its value depends only on the distance between x and a center c.

In our case:

$$\phi(x_i, c_j) = \exp\left(-\frac{\|x_i - c_j\|^2}{2\sigma^2}\right).$$

The term "radial" refers to the fact that the function is isotropic—it depends only on the radius (distance) and not on direction.

Why RBFs are used Using an RBF provides the following properties:

- **Smooth decay:** The influence of a centroid decreases smoothly with distance, avoiding sharp boundaries.
- **Locality:** Only nearby centroids contribute significantly, preserving local geometric structure.
- **Continuity:** The resulting mapping from vectors to attributes is continuous, ensuring that similar vectors receive similar attribute values.

These properties are essential for constructing metadata that is meaningfully correlated with the geometry of the embedding space. In particular, the RBF weighting ensures that the resulting attributes respect the underlying notion of similarity induced by the vector representation.

4.2.6 Continuous Attribute Construction

We define the continuous attribute value for each vector as:

$$a_i^{(\text{cont})} = \frac{\sum_{j=1}^{k} w_{ij} v_j}{\sum_{j=1}^{k} w_{ij}}.$$

This corresponds to a kernel-weighted aggregation of cluster-level signals.

Marginal Flattening To reduce skew in the attribute distribution, we apply a rank-based rescaling to $a_i^{(\text{cont})}$.

Specifically, we replace each value by its empirical cumulative distribution function (CDF) rank:

$$a_i^{(\text{cont})} \leftarrow \frac{\text{rank}(a_i^{(\text{cont})})}{N}.$$

This transformation preserves ordering while producing a more uniform marginal distribution, improving the stability of neighbor comparisons.

Normalization and Quantization We normalize:

$$\hat{a}_i = \frac{a_i^{(\text{cont})} - \min_i a_i^{(\text{cont})}}{\max_i a_i^{(\text{cont})} - \min_i a_i^{(\text{cont})}},$$

with appropriate handling for degenerate cases.

Finally, we quantize:

$$a_i = \text{round}(200 \cdot \hat{a}_i), \quad a_i \in \{0, \ldots, 200\}.$$

This yields discrete attributes suitable for range-based filtering.

4.3 Uncorrelated Attribute Generation

4.4 Query Range Generation

4.4.1 Positive (correlated)

4.4.2 Negative (anti-correlated)

4.4.3 Random

4.5 Selectivity Control

4.6 Dataset Validation

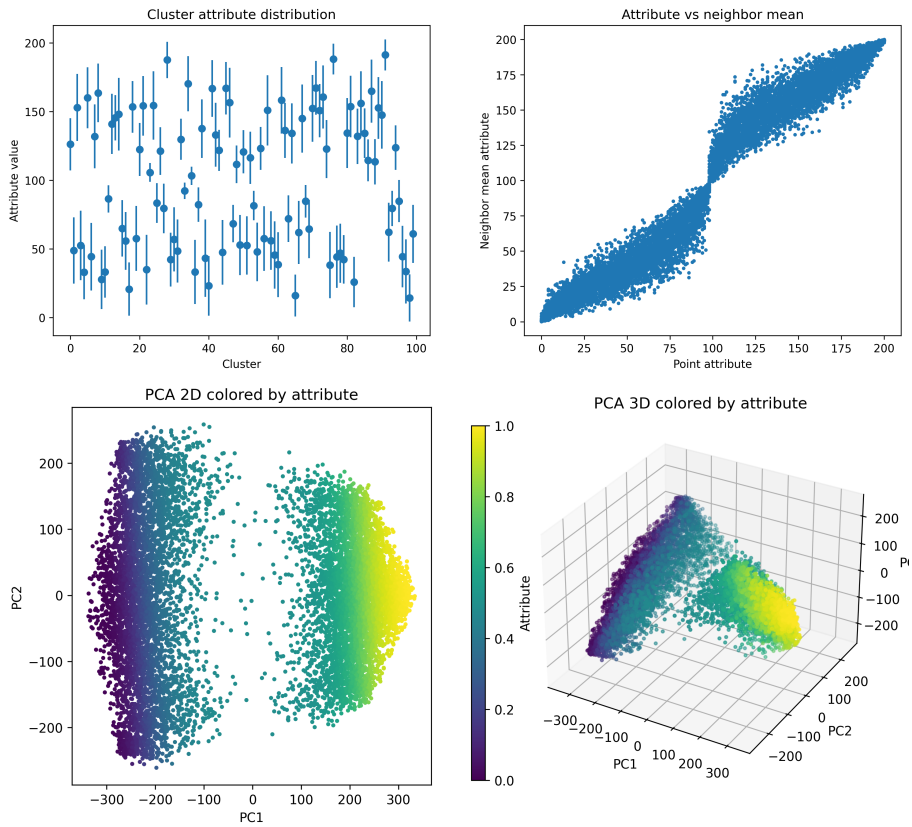

Figure 4.1: Your caption describing the image.

4.6.1 Neighbor consistency

4.6.2 Cluster distributions

5. Correlation Metric

5.1 Local vs Global Attribute Distributions

5.2 Cluster-based Approximation

5.3 Jensen–Shannon Divergence

5.4 Lift (range alignment)

5.5 Combined Score & Interpretation

6. System & Execution Strategies

6.1 Pre-filtering

6.2 Post-filtering

6.3 Inline Filtering

6.4 Implementation Details

7. Experimental Setup

7.1 Offline computed statistics

7.2 Evaluation Metrics

7.3 Ground Truth Computation

8. Results & Discussion

Bibliography

[1] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni, *Navigable proximity graph-driven native hybrid queries with structured and unstructured constraints*, 2022. arXiv: 2203.13601 [cs.DB]. [Online]. Available: https://arxiv.org/abs/2203.13601.

[2] J. Wang, X. Yi, R. Guo, *et al.*, "Milvus: A purpose-built vector data management system," in *Proceedings of the 2021 International Conference on Management of Data*, ser. SIGMOD '21, Virtual Event, China: Association for Computing Machinery, 2021, 2614–2627, ISBN: 9781450383431. DOI: 10.1145/3448016.3457550. [Online]. Available: https://doi.org/10.1145/3448016.3457550.

[3] X. Xu, C. Li, Y. Wang, and Y. Xia, "Multiattribute approximate nearest neighbor search based on navigable small world graph," *Concurrency and Computation: Practice and Experience*, vol. 32, no. 24, e5970, 2020. DOI: https://doi.org/10.1002/cpe.5970. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpe.5970. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.5970.

[4] S. Gollapudi, N. Karia, V. Sivashankar, *et al.*, "Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters," in *Proceedings of the ACM Web Conference 2023*, ser. WWW '23, Austin, TX, USA: Association for Computing Machinery, 2023, 3406–3416, ISBN: 9781450394161. DOI: 10.1145/3543507.3583552. [Online]. Available: https://doi.org/10.1145/3543507.3583552.

[5] F. Almeida and G. Xexéo, *Word embeddings: A survey*, 2023. arXiv: 1901.09069 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1901.09069.

[6] L. Ma, R. Zhang, Y. Han, *et al.*, *A comprehensive survey on vector database: Storage and retrieval technique, challenge*, 2026. arXiv: 2310.11703 [cs.DB]. [Online]. Available: https://arxiv.org/abs/2310.11703.

[7] S. Arya, D. M. Mount, N. S. Netanyahu, R. Silverman, and A. Y. Wu, "An optimal algorithm for approximate nearest neighbor searching fixed dimensions," *J. ACM*, vol. 45, no. 6, 891–923, Nov. 1998, ISSN: 0004-5411. DOI: 10.1145/293347.293348. [Online]. Available: https://doi.org/10.1145/293347.293348.

[8] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, ser. STOC '98, Dallas, Texas, USA: Association for Computing Machinery, 1998, 604–613, ISBN: 0897919629. DOI: 10.1145/276698.276876. [Online]. Available: https://doi.org/10.1145/276698.276876.

[9] O. Jafari, P. Maurya, P. Nagarkar, K. M. Islam, and C. Crushev, *A survey on locality sensitive hashing algorithms and their applications*, 2021. arXiv: 2102.08942 [cs.DB]. [Online]. Available: https://arxiv.org/abs/2102.08942.

[10] A. Gionis, P. Indyk, and R. Motwani, "Similarity search in high dimensions via hashing," in *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*, Edinburgh, Scotland, 1999, pp. 518–529. [Online]. Available: https://www.cs.princeton.edu/courses/archive/spring13/cos598C/Gionis.pdf.

[11] Y. A. Malkov and D. A. Yashunin, *Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs*, 2018. arXiv: 1603.09320 [cs.DS]. [Online]. Available: https://arxiv.org/abs/1603.09320.

[12] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, "Diskann: Fast accurate billion-point nearest neighbor search on a single node," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32, Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Paper.pdf.

[13] O. Salemohamed, L. Charlin, S. Garg, V. Sharan, and G. Valiant, *Discovering data structures: Nearest neighbor search and beyond*, 2024. arXiv: 2411.03253 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2411.03253.

[14] C.-Y. Chiu, A. Prayoonwong, and Y.-C. Liao, *Learning to index for nearest neighbor search*, 2019. arXiv: 1807.02962 [cs.IR]. [Online]. Available: https://arxiv.org/abs/1807.02962.

[15] M. Douze, A. Guzhva, C. Deng, *et al.*, *The faiss library*, 2025. arXiv: 2401.08281 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2401.08281.

[16] J. Johnson, M. Douze, and H. Jégou, *Billion-scale similarity search with gpus*, 2017. arXiv: 1702.08734 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1702.08734.

[17] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, *Acorn: Performant and predicate-agnostic search over vector embeddings and structured data*, 2024. arXiv: 2403.04871 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2403.04871.

[18] Y. Cai, J. Shi, Y. Chen, and W. Zheng, "Navigating labels and vectors: A unified approach to filtered approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 6, Dec. 2024. DOI: 10.1145/3698822. [Online]. Available: https://doi.org/10.1145/3698822.

[19] Y. Chronis, H. Caminal, Y. Papakonstantinou, F. Özcan, and A. Ailamaki, "Filtered vector search: State-of-the-art and research opportunities," *Proc. VLDB Endow.*, vol. 18, no. 12, 5488–5492, Aug. 2025, ISSN: 2150-8097. DOI: 10.14778/3750601.3750700. [Online]. Available: https://doi.org/10.14778/3750601.3750700.

[20] L. Getoor, B. Taskar, and D. Koller, "Selectivity estimation using probabilistic models," *SIGMOD Rec.*, vol. 30, no. 2, 461–472, May 2001, ISSN: 0163-5808. DOI: 10.1145/376284.375727. [Online]. Available: https://doi.org/10.1145/376284.375727.

[21] C. A. Lynch, "Selectivity estimation and query optimization in large databases with highly skewed distribution of column values," in *Very Large Data Bases Conference*, 1988. [Online]. Available: https://api.semanticscholar.org/CorpusID:20418479.

[22] H. Zhang, C. Mayer, M. A. Khamis, D. Olteanu, and D. Suciu, *Lpbound: Pessimistic cardinality estimation using ℓ_p-norms of degree sequences*, 2025. arXiv: 2502.05912 [cs.DB]. [Online]. Available: https://arxiv.org/abs/2502.05912.

A. First Appendix

B. Second Appendix